

CATALOGUE FORMATIONS IT

PLAN DE FORMATION

DevOps, CI/CD & Orchestration

Docker • Kubernetes • Docker Swarm • Pipelines CI/CD

 DURÉE
4 heures minimum

 PUBLIC CIBLE
Dev Backend / Frontend / Mobile

 NIVEAU
Intermédiaire à Avancé

 FORMAT
Présentiel / Distanciel

1. IDENTIFICATION DE LA FORMATION

Intitulé de la formation	DevOps, CI/CD & Orchestration — Docker, Kubernetes, Docker Swarm, Pipelines
Code formation	IT-DEVOPS-CICD-001
Domaine	Technologies de l'information — Développement & Opérations (DevOps)
Durée totale	4 heures (240 minutes)
Nombre de modules	7 modules
Modalité	Présentiel ou distanciel (visioconférence avec partage d'écran)
Langue	Français

2. CONTEXTE ET OBJECTIFS PÉDAGOGIQUES

2.1 Contexte et enjeux

Cette formation est conçue pour permettre aux équipes de développement de maîtriser les processus de déploiement modernes et l'automatisation des pipelines. Dans un contexte de transformation digitale où le time-to-market est déterminant, la culture DevOps et la maîtrise des outils d'orchestration constituent des compétences essentielles pour tout développeur.

2.2 Objectifs généraux

- Comprendre la culture et les principes fondamentaux DevOps (modèle CALMS)
- Maîtriser la conteneurisation avec Docker (images, conteneurs, Compose)
- Orchestrer des conteneurs avec Kubernetes et Docker Swarm
- Mettre en place des pipelines CI/CD automatisés (GitLab CI, GitHub Actions)
- Gérer les environnements de déploiement : développement, staging et production
- Implémenter le monitoring et la gestion centralisée des logs

2.3 Objectifs opérationnels (compétences attendues en fin de formation)

À l'issue de la formation, le participant sera capable de :

#	Compétence visée	Niveau attendu
C1	Expliquer la philosophie DevOps et le modèle CALMS	<i>Maîtrise — Savoir expliquer</i>
C2	Créer et optimiser des images Docker (Dockerfile multi-stage)	<i>Maîtrise — Savoir faire</i>
C3	Orchestrer des services multi-conteneurs avec Docker Compose	<i>Maîtrise — Savoir faire</i>
C4	Déployer une application sur un cluster Kubernetes (kubectl, YAML)	<i>Application — Savoir faire</i>
C5	Configurer un pipeline CI/CD sur GitHub Actions ou GitLab CI	<i>Application — Savoir faire</i>
C6	Choisir et appliquer une stratégie de déploiement (Blue/Green, Canary)	<i>Compréhension — Savoir choisir</i>
C7	Mettre en place un monitoring avec Prometheus/Grafana et centraliser les logs (ELK)	<i>Sensibilisation — Savoir utiliser</i>

3. PUBLIC CIBLE ET PRÉREQUIS

Public cible	Prérequis obligatoires
› Développeurs Backend (Node.js, Python, Java...) › Développeurs Frontend (React, Vue, Angular...) › Développeurs Mobile souhaitant intégrer les pipelines CI/CD › Tech Leads et développeurs seniors › Toute personne souhaitant monter en compétences DevOps	› Maîtrise d'un langage de programmation (niveau intermédiaire) › Connaissance de base de Git (commit, push, pull request) › Notions de ligne de commande Linux/Bash › Compréhension des concepts client/serveur et API REST Prérequis recommandés : › Expérience sur un projet en production › Notions de YAML

4. PROGRAMME DÉTAILLÉ ET DÉCOUPAGE HORAIRE

#	Module / Contenu	Durée	Format	Horaire indicatif
01	Fondamentaux DevOps Culture DevOps & philosophie CALMS, CI vs CD vs Continuous Deployment, avant/après DevOps, quiz de validation	30 min	Cours magistral	H+00 → H+30
02	Docker & Conteneurisation Images, conteneurs, Dockerfile multi-stage, Docker Compose, volumes, réseaux.	45 min	Cours	H+30 → H+1h15
—	Pause — Bilan intermédiaire module 1 & 2	10 min	—	H+1h15 → H+1h25
03	Kubernetes & Docker Swarm Architecture K8s, objets essentiels (Pod, Deployment, Service, ConfigMap, Secret, Ingress), Docker Swarm vs K8s.	60 min	Cours	H+1h25 → H+2h25
04	Pipelines CI/CD Anatomie d'un pipeline, GitHub Actions (workflows, triggers, jobs, runners), GitLab CI (stages, artifacts, environments).	45 min	Cours	H+2h25 → H+3h10
—	Pause — Q&A mi-parcours	10 min	—	H+3h10 → H+3h20
05	Gestion des environnements Dev / Staging / Production, stratégies : Blue/Green, Canary Release, Rolling Update	30 min	Cours	H+3h20 → H+3h50
06	Monitoring & Logs Prometheus, Grafana, stack ELK, Golden Signals (latency, traffic, errors, saturation), bonnes pratiques de logging structuré	30 min	Cours	H+3h50 → H+4h20
TOTAL (hors pauses)		4h20		

* La durée minimale de 4h est garantie. Le programme peut être condensé à 4h en réduisant les ateliers ou en fusionnant les modules 5 et 6.

5. MÉTHODES ET MOYENS PÉDAGOGIQUES

📖 Apports théoriques	🔧 Mises en pratique	💬 Échanges & interactions
› Cours magistraux illustrés › Slides visuels et schémas d'architecture › Comparatifs technologiques (VM vs conteneurs, K8s vs Swarm) › Démonstrations en direct (live coding)	› 4 ateliers progressifs guidés › Projet final intégrateur (60 min) › Environnement sandbox prêt à l'emploi › Dépôts Git fournis comme point de départ	› Quiz de validation (Module 1) › Questions/Réponses après chaque module › Retours d'expérience des participants › Partage de bonnes pratiques

Moyens techniques requis

Côté formateur	Côté participant
› Ordinateur avec Docker Desktop installé › Minikube / kubectl configurés › Compte GitHub et GitLab actifs › Vidéoprojecteur ou partage d'écran › Dépôt de code de référence accessible	› Ordinateur avec accès administrateur › Docker Desktop installé (version récente) › Éditeur de code (VS Code recommandé) › Compte GitHub actif › Connexion internet stable

6. RÉCAPITULATIF — CE QUE VOUS SAUREZ FAIRE À L'ISSUE DE LA FORMATION

01	Comprendre et expliquer la philosophie DevOps, la culture CALMS et les bénéfices du CI/CD
02	Conteneuriser une application avec Docker (Dockerfile, images, volumes, réseaux)
03	Orchestrer des services multi-conteneurs avec Docker Compose en environnement local
04	Déployer et gérer une application sur un cluster Kubernetes (Deployments, Services, ConfigMaps)
05	Créer et configurer des pipelines CI/CD automatisés avec GitHub Actions ou GitLab CI
06	Gérer les environnements dev/staging/production et appliquer les stratégies de déploiement
07	Mettre en place le monitoring (Prometheus/Grafana) et la gestion centralisée des logs (ELK)